Python & Data Science Notes

---

1. Environment Setup

It is always a good practice to start with creating a separate environment.

Three Ways to Create Virtual Environments

1. Using `venv` Module

```
python -m venv venv_name
venv_name\Scripts\activate      # Activate
deactivate                       # Deactivate
```

2. Using `virtualenv` (Linux / PowerShell / Git Bash)

```
pip install virtualenv
virtualenv -p python3 venv_name
venv_name\Scripts\activate      # Activate
deactivate                       # Deactivate
```

3. Using Anaconda *(Recommended)*

```
conda create -p venv_name python==3.14 -y    # Can use other versions
conda activate venv/                          # Activate
conda deactivate                              # Deactivate
```

Setting Up with Anaconda (Step-by-Step)

- Open command prompt / terminal in VS Code
- Run: `conda create -p venv python==3.14`
- Type `y` to confirm → environment created
- Add a `requirements.txt` file
- Add package names inside `requirements.txt`
- Activate: `conda activate venv/`
- Install packages from file: `pip install -r requirements.txt`
- **Jupyter Notebook** (`.ipynb`) requires two packages: `ipykernel` + Jupyter extension
- Click on "detecting kernels" in `.ipynb` file and select the created environment

---

2. Python Modules and Packages

Helps organize and reuse code.

Importing Modules

```python
# Way 1 - import the whole module
import math
math.sqrt(16)

# Way 2 - import specific functions
from math import sqrt, pi
print(sqrt(16))
print(sqrt(25))
print(pi)

# Importing everything
from math import *
print(sqrt(16))
print(pi)

# Using an alias
import numpy as np     # np is the alias
np.array([1, 2, 3])    # access numpy with np.
```

Creating Your Own Custom Module / Package

- Make a folder (e.g. `package/`)
- Inside it, create a special file: **`__init__.py`** (makes Python treat the folder as a package)
- Create your module file inside: `package/maths.py`
- Define your functions inside that file

```python
# Calling your custom module
from package.maths import addition
addition(2, 3)

# OR
from package import maths
maths.addition(2, 3)
```

Syntax:

```
from folder_name.package_name import function_name
```

> **Note:** For subpackages (nested folders), place an `__init__.py` in each subfolder too.

```python
from folder_name.subpackageFolder_name.packagename import function_name
function_name()
```

---

3. Python Standard Library Overview

In Python, the standard library is a big collection of built-in modules that come with Python itself — no extra installs needed. Think of it as Python's **"batteries included"** toolbox.

`array` — Basic Arrays

```
import array
arr = array.array('i', [1, 2, 3, 4])
print(arr)
```

`math` — Mathematical Functions

```
import math
math.sqrt(16)
```

`random` — Random Values

```
import random
print(random.randint(1, 10))
print(random.choice(['apple', 'banana', 'cherry']))
```

`os` — Operating System / IO Operations

```
import os
print(os.getcwd())
os.mkdir('test_dir')
```

`shutil` — High-Level File Operations

```
import shutil
shutil.copyfile('source.txt', 'destination.txt')
```

`json` — Data Serialization (used in APIs)

```
import json

data = {'name': 'Sher', 'age': 25}
json_str = json.dumps(data)        # Dict → JSON string
print(json_str)
print(type(json_str))

parsed_data = json.loads(json_str) # JSON string → Dict
print(parsed_data)
print(type(parsed_data))
```

Result:

```
{"name": "Sher", "age": 25}
<class 'str'>
{'name': 'Sher', 'age': 25}
<class 'dict'>
```

`csv` — Working with CSV Files

```python
import csv

with open('example.csv', mode='w', newline='') as file:
    writer = csv.writer(file)
    writer.writerow(['name', 'age'])
    writer.writerow(['Sher', '20'])

with open('example.csv', mode='r') as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

`datetime` — Date and Time

```python
from datetime import datetime, timedelta    # timedelta is used for time changes

now = datetime.now()
print(now)

yesterday = now - timedelta(days=1)
print(yesterday)
```

`time` — Time / Sleep

```python
import time
print(time.time())
time.sleep(2)          # Program sleeps for 2 seconds
print(time.time())
```

`re` — Regular Expressions

`re` lets you search, match, split, and replace text using patterns (regex).

```python
import re

pattern = r'\d+'
text = 'There are 212 apples'
match = re.search(pattern, text)
print(match.group())
```

Result:

```
212
```

F-Strings

```python
x = 10
print(f"my score is {x}")
```

---

4. Working with File Paths

Listing Directory Contents

```python
import os
items = os.listdir('.')
print(items)
```

Joining Paths

```python
import os

dir_name = "folder"
file_name = "file.txt"
full_path = os.path.join(dir_name, file_name)
full_final_path = os.path.join(os.getcwd(), dir_name, file_name)
print(full_path)
print(full_final_path)
```

Checking if a File Exists

```python
import os

path = 'example1.txt'
if os.path.exists(path):
    print(f"The path '{path}' exists")
else:
    print(f"The path '{path}' does not exist")
```

Checking if Path is a File or Directory

```python
import os

path = 'Notes.txt'
if os.path.isfile(path):
    print(f"'{path}' is a file")
elif os.path.isdir(path):
    print(f"'{path}' is a directory")
```

```
else:
    print(f"'{path}' is neither a directory nor a file")
```

Getting the Absolute Path

```
relative_path = 'Notes.txt'
absolute_path = os.path.abspath(relative_path)
print(absolute_path)
```

> **Note:** This is better than manually joining with `os.path.join(os.getcwd(), ...)`.

---

5. Exception Handling

Basic `try-except` Block

```
try:
    a = b
except:
    print("Variable not assigned")
```

Displaying the Error Type

```
try:
    a = b
except NameError as ex:
    print(ex)
```

Syntax:

```
try:
    # code
except ErrorType as alias:
    print(alias)
```

Catching Specific Exceptions

```
# ZeroDivisionError
try:
    result = 1 / 0
except ZeroDivisionError as err:
    print("Enter denom greater than 0")
```

Multiple `except` Blocks

> **Note:** All error types derive from the base `Exception` class — always put `Exception` **last**.

```python
try:
    numb = int(input("Enter a value: "))
    res = 10 / numb
except ValueError:
    print("Enter valid value")
except ZeroDivisionError:
    print("Enter greater than 0")
except Exception as ex:
    print(ex)
```

`try`, `except`, and `else`

`else` executes **only if** `try` runs successfully without triggering any `except` block.

```python
try:
    numb = int(input("Enter a value: "))
    res = 10 / numb
except ValueError:
    print("Enter valid value")
except ZeroDivisionError:
    print("Enter greater than 0")
except Exception as ex:
    print(ex)
else:
    print(f"The value is {res}")
```

`try`, `except`, `else`, and `finally`

`finally` executes **no matter what** — whether the `try` succeeded or an exception was caught.

```python
try:
    numb = int(input("Enter a value: "))
    res = 10 / numb
except ValueError:
    print("Enter valid value")
except ZeroDivisionError:
    print("Enter greater than 0")
except Exception as ex:
    print(ex)
else:
    print(f"The value is {res}")
finally:
    print("Execution completed")
```

---

6. NumPy

NumPy is a fundamental library for scientific computing in Python. It provides support for arrays and matrices, along with a collection of mathematical functions to operate on these data structures.

```
import numpy as np
```

Creating Arrays

1D Array

```
arr1 = np.array([1, 2, 3, 4, 5])
print(arr1)
print(type(arr1))       # Not a list!
print(arr1.shape)       # (5,) - no rows/columns in 1D
```

Reshaping a 1D Array to 2D

```
arr2 = np.array([1, 2, 3, 4, 5])
arr2.reshape(1, 5)      # 1 row, 5 columns → array([[1, 2, 3, 4, 5]])
```

Two `[[]]` brackets = 2D array, three `[[[]]]` = 3D array.

2D and 3D Arrays

```
arr3 = np.array([[1, 2, 3, 4, 5]])           # 2D - one row
arr4 = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])  # 2D - two rows
print(arr4.shape)    # (2, 4) → 2 rows, 4 columns
```

Inbuilt Array Creation Functions

```
np.arange(0, 10, 2)              # [0, 2, 4, 6, 8]
np.arange(0, 10, 2).reshape(5, 1)  # Reshape to 5 rows, 1 column

np.ones((3, 4))    # 3x4 array of 1s
np.eye(3)          # 3x3 identity matrix
```

Array Attributes

```
arr = np.array([[1, 2, 3], [4, 5, 6]])

print("Array:\n", arr)
print("Shape:", arr.shape)                    # (2, 3)
print("Number of dimensions:", arr.ndim)      # 2
print("Size (number of elements):", arr.size) # 6
print("Data type:", arr.dtype)                # int64
print("Item size (in bytes):", arr.itemsize)  # 8
```

Vectorized Operations

```
arr1 = np.array([1, 2, 3, 4, 5])
arr2 = np.array([10, 20, 30, 40, 50])

print("Addition:", arr1 + arr2)
print("Subtraction:", arr2 - arr1)
# Similarly: multiplication, division, etc.
```

Universal Functions

Functions that apply to the **entire array** at once.

```
arr = np.array([2, 3, 4, 5, 6])

print(np.sqrt(arr))   # Square root
print(np.exp(arr))    # Exponential
print(np.sin(arr))    # Sine
print(np.log(arr))    # Natural log
```

Array Slicing and Indexing

```
arr = np.array([[1, 2, 3, 4],
                [5, 6, 7, 8],
                [9, 10, 11, 12]])

arr[0][0]          # Single element → 1
arr[1:, 2:]        # Pick 7, 8, 11, 12
arr[0:2, 2:4]      # Pick 3, 4, 7, 8
```

Modifying Array Elements

```
arr[0, 0] = 100        # Modify single element
arr[1:] = 99           # Set all rows from index 1 to 99
arr[1:3, 2:4] = 34     # Modify a slice
```

Statistical Concepts — Normalization

```
data = np.array([1, 2, 3, 4, 5])

mean = np.mean(data)
std_dev = np.std(data)
normalized_data = (data - mean) / std_dev

print(normalized_data)
```

Statistical Operations

```
data = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

print("Mean:", np.mean(data))
print("Median:", np.median(data))
print("Standard Deviation:", np.std(data))
print("Variance:", np.var(data))
```

Logical Operations

```
data = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

data[(data >= 5) & (data <= 8)]     # Elements between 5 and 8
# Any logical operation can be used: &, |, ~
```

---

7. Pandas — Data Manipulation Library

Pandas is a powerful data manipulation library in Python, widely used for data analysis and cleaning. It provides two primary data structures:

- **Series** — One-dimensional array-like object
- **DataFrame** — Two-dimensional, size-mutable, tabular data structure with labeled axes (rows and columns)

```
import pandas as pd
```

7.1 Pandas Series

A Pandas Series is a one-dimensional array-like object that can hold any data type. Similar to a column in a table.

From a List

```
data = [1, 2, 3, 4, 5]
series = pd.Series(data)
print("Series \n", series)
print(type(series))
```

Result:

```
0    1
1    2
2    3
3    4
4    5
```

```
dtype: int64
<class 'pandas.core.series.Series'>
```

From a Dictionary

```
data = {'a': 1, 'b': 2, 'c': 3}
series_dict = pd.Series(data)
print(series_dict)
```

Result:

```
a    1
b    2
c    3
dtype: int64
```

Keys become the index, values are the series values.

With a Custom Index

```
data = [10, 20, 30]
index = ['a', 'b', 'c']
pd.Series(data, index=index)
```

Result:

```
a    10
b    20
c    30
dtype: int64
```

7.2 Pandas DataFrame

A Pandas DataFrame is a two-dimensional, size-mutable, tabular data structure with labeled axes.

Reading Data

Method	Use
`pd.read_csv()`	CSV or delimited text files
`pd.read_excel()`	`.xlsx` / `.xls` files (supports multiple sheets)
`pd.read_json()`	Web-based JSON data
`pd.read_sql()`	SQL databases (PostgreSQL, MySQL, SQLite)

Creating a DataFrame from a Dictionary of Lists

```
data = {
    'Name': ['Sher', 'Hassan', 'Khan'],
    'Age':  [18, 19, 20],
    'City': ['Quetta', 'Islamabad', 'Zhob']
}
df = pd.DataFrame(data)
print(df)
```

Result:

```
     Name  Age       City
0    Sher   18     Quetta
1  Hassan   19  Islamabad
2    Khan   20       Zhob
```

Creating a DataFrame from a List of Dictionaries

```
data1 = [
    {'Name': 'Sher',   'Age': 20, 'City': 'Quetta'},
    {'Name': 'Hassan', 'Age': 20, 'City': 'Islamabad'},
    {'Name': 'Khan',   'Age': 20, 'City': 'Zhob'},
    {'Name': 'Ali',    'Age': 20, 'City': 'Karachi'}
]
df1 = pd.DataFrame(data1)
print(df1)
```

Result:

```
     Name  Age       City
0    Sher   20     Quetta
1  Hassan   20  Islamabad
2    Khan   20       Zhob
3     Ali   20    Karachi
```

Reading a CSV File

```
df = pd.read_csv('sales_data.csv')
print(df.head(5))    # First 5 rows
print(df.tail(5))    # Last 5 rows
```

7.3 Accessing Data from a DataFrame

```
df['Name']              # Access a column (returns Series)
df.loc[0]               # Access row by label/index
df.iloc[0]              # Access row by position
df.loc[0, 'Name']       # Specific element by label
```

```
df.iloc[0, 2]           # Specific element by position
df.at[1, 'Age']         # Element with .at (label-based)
df.iat[1, 2]            # Element with .iat (position-based)
```

7.4 Data Manipulation with DataFrame

Adding a New Column

```
df['Salary'] = [50000, 60000, 70000, 80000]
```

Removing a Column

```
df.drop('Salary', axis=1)             # axis=1 for columns
df.drop(columns=['Salary'])           # Alternative
df = df.drop(columns=['Salary'])      # Save changes
df.drop('Salary', axis=1, inplace=True)  # In-place
```

Note: Default `axis=0` removes rows, not columns!

Modifying Column Values

```
df['Age'] = df['Age'] + 1
```

Removing a Row

```
df.drop(3, inplace=True)    # Remove row at index 3
```

Statistical Summary

```
df.describe()
```

Result:

```
         Age    Salary
count    3.0     3.0
mean    21.0  60000.0
std      1.0  10000.0
min     20.0  50000.0
25%     20.5  55000.0
50%     21.0  60000.0
75%     21.5  65000.0
max     22.0  70000.0
```

7.5 Data Manipulation and Analysis

Checking Data Types

```
df.dtypes
```

Handling Missing Values

```
df.isnull()                # True where entries are missing
df.isnull().any(axis=1)    # Which rows have missing values
df.isnull().sum()          # Count of missing values per column
```

Result:

```
Unit Price    4
...all others: 0
```

Filling Missing Values

```
df.fillna(0)     # Fill all NaN with 0

# Fill with the column mean
df['Unit_Price_fillNA'] = df['Unit Price'].fillna(df['Unit Price'].mean())
```

Renaming Columns — `.rename()`

```
# Syntax: df.rename(columns={'OldName': 'NewName'})
df = df.rename(columns={'Country': 'Nationality'})
```

Changing Data Types — `.astype()`

```
# Convert Units Sold from int to float
df['New Units Sold'] = df['Units Sold'].astype(float)
```

Applying Functions — `.apply()`

```
df['New Value'] = df['New Units Sold'].apply(lambda x: x * 2)
```

7.6 Data Aggregation and Grouping

Group by One Column

```
grouped_mean = df.groupby('Region')['Unit Cost'].mean()
print(grouped_mean)
```

Result:

```
Region
Asia                                239.587273
Australia and Oceania               154.744545
```

```
Central America and the Caribbean    157.817143
Europe                               223.166364
Middle East and North Africa         152.450000
North America                        205.293333
Sub-Saharan Africa                   183.677500
Name: Unit Cost, dtype: float64
```

Group by Two Columns

```
grouped_sum = df.groupby(['Region', 'Item Type'])['Unit Cost'].sum()
print(grouped_sum)
```

Aggregate Multiple Functions

```
grouped_agg = df.groupby('Region')['Unit Cost'].agg(['mean', 'sum', 'count'])
print(grouped_agg)
```

Result:

```
                                          mean       sum  count
Region
Asia                                239.587273   2635.46     11
Australia and Oceania               154.744545   1702.19     11
Central America and the Caribbean   157.817143   1104.72      7
Europe                              223.166364   4909.66     22
Middle East and North Africa        152.450000   1524.50     10
North America                       205.293333    615.88      3
Sub-Saharan Africa                  183.677500   6612.39     36
```

7.7 Merging and Joining DataFrames

```
df1 = pd.DataFrame({'Key': ['A', 'B', 'C'], 'Value1': [1, 2, 3]})
df2 = pd.DataFrame({'Key': ['A', 'B', 'D'], 'Value1': [4, 5, 6]})
```

Inner Join — only matching keys

```
pd.merge(df1, df2, on="Key", how="inner")
```

Result:

```
  Key  Value1_x  Value1_y
0   A         1         4
1   B         2         5
```

Outer Join — all keys, NaN where no match

```
pd.merge(df1, df2, on="Key", how="outer")
```

Result:

```
  Key  Value1_x  Value1_y
0   A       1.0       4.0
1   B       2.0       5.0
2   C       3.0       NaN
3   D       NaN       6.0
```

Left Join — all keys from left DataFrame

```
pd.merge(df1, df2, on="Key", how="left")
```

Result:

```
  Key  Value1_x  Value1_y
0   A         1       4.0
1   B         2       5.0
2   C         3       NaN
```

Right Join — all keys from right DataFrame

```
pd.merge(df1, df2, on="Key", how="right")
```

Result:

```
  Key  Value1_x  Value1_y
0   A       1.0         4
1   B       2.0         5
2   D       NaN         6
```

---

8. Reading Data From Different Sources

8.1 Reading Data from JSON — `.read_json()` and `.StringIO()`

The flow is: **Read Data From JSON → Convert to String → Store**

```
import pandas as pd
from io import StringIO

Data = '{"employee_name": "James", "email": "james@gmail.com", "job_profile": [{"title1":"Te
df = pd.read_json(StringIO(Data))
df
```

Result:

```
  employee_name            email               job_profile
0         James  james@gmail.com   {'title1': 'Team Lead'}
```

```
1         James  james@gmail.com  {'title2': 'Sr. Developer'}
```

8.1.1 Converting a DataFrame Back to JSON — `.to_json()`

```
df.to_json()
```

Result:

```
'{"employee_name":{"0":"James","1":"James"},"email":{"0":"james@gmail.com","1":"james@g
"job_profile":{"0":{"title1":"Team Lead"},"1":{"title2":"Sr. Developer"}}}'
```

8.1.2 The `orient` Parameter in `.to_json()` The `orient` parameter determines the **format and structure** of the resulting JSON string.

Since JSON is a flexible format, there are multiple ways to represent a table (rows and columns) in a nested string. Changing the `orient` changes how keys and values are paired up.

8.1.2.1 Common `orient` Options

Orient	Description	Best Use Case
`'records'`	A list of dictionaries: `[{column -> value}, ...]`	Web APIs — most frontend frameworks expect this
`'split'`	A dictionary containing index, columns, and data	Preserve exact structure without repeating columns
`'index'`	A dictionary of dictionaries: `{index -> {col:val}}`	Quick lookups by a specific ID or index
`'columns'`	A dictionary of dictionaries: `{col -> {idx:val}}`	Default — optimized for column-based access
`'values'`	Just the data in a nested list (no labels)	Minimal file size when labels aren't needed
`'table'`	Includes a JSON schema (data types and index info)	High-integrity data transfer between pandas

8.1.2.2 Code Examples

```python
import pandas as pd
```

```
df = pd.DataFrame({
    'Name': ['Alice', 'Bob'],
    'Age':  [25, 30]
})

# 1. records - most common for web apps
print(df.to_json(orient='records'))
```

Result:

```
[{"Name":"Alice","Age":25},{"Name":"Bob","Age":30}]
```

```
# 2. split - separates metadata from data
print(df.to_json(orient='split'))
```

Result:

```
{"columns":["Name","Age"],"index":[0,1],"data":[["Alice",25],["Bob",30]]}
```

```
# 3. index - uses row index as the primary key
print(df.to_json(orient='index'))
```

Result:

```
{"0":{"Name":"Alice","Age":25},"1":{"Name":"Bob","Age":30}}
```

When to use which? - Sending data to a **JavaScript app** → almost always use `orient='records'` - **Saving to disk** to reopen in pandas later → use `orient='split'` or `'table'` (preserves index and column names more reliably)

8.1.2.3 Applying `orient` on Our DataFrame

```
df.to_json(orient="index")
```

Result:

```
'{"0":{"employee_name":"James","email":"james@gmail.com","job_profile":{"title1":"Team
  "1":{"employee_name":"James","email":"james@gmail.com","job_profile":{"title2":"Sr. D
```

```
df.to_json(orient="records")
```

Result:

```
'[{"employee_name":"James","email":"james@gmail.com","job_profile":{"title1":"Team Lead
  {"employee_name":"James","email":"james@gmail.com","job_profile":{"title2":"Sr. Devel
```

8.2 Reading Data From a URL — `.read_csv(url)`

```
df = pd.read_csv(
    "https://archive.ics.uci.edu/ml/machine-learning-databases/wine/wine.data",
    header=None
```

```
)
df.head()
```

Result:

```
   0      1     2     3     4    5     6     7     8     9     10    11    12    13
0  1  14.23  1.71  2.43  15.6  127  2.80  3.06  0.28  2.29  5.64  1.04  3.92  1065
1  1  13.20  1.78  2.14  11.2  100  2.65  2.76  0.26  1.28  4.38  1.05  3.40  1050
2  1  13.16  2.36  2.67  18.6  101  2.80  3.24  0.30  2.81  5.68  1.03  3.17  1185
3  1  14.37  1.95  2.50  16.8  113  3.85  3.49  0.24  2.18  7.80  0.86  3.45  1480
4  1  13.24  2.59  2.87  21.0  118  2.80  2.69  0.39  1.82  4.32  1.04  2.93   735
```

Note: `header=None` is used when the data file does **not** contain column names in the first row. Without it, pandas will automatically treat the first row of data as the column headers.

8.2.1 Converting Back to CSV

```
df.to_csv("wine.csv")
# New file wine.csv is created and stored with the df data
```

8.3 Reading Data From HTML — `.read_html()`

```
url = "https://www.fdic.gov/bank/individual/failed/banklist.html"
df = pd.read_html(url)

df[0]    # df[0] gives you the first table extracted from the website
```

Result:

```
                                Bank Name       City      State   Cert  ...  Closing Date
0   Metropolitan Capital Bank & Trust    Chicago   Illinois  57488  ...  January 30, 20
1       The Santa Anna National Bank  Santa Anna      Texas   5520  ...   June 27, 2025
2              Pulaski Savings Bank    Chicago   Illinois  28611  ...  January 17, 2025
...
24    Seaway Bank and Trust Company    Chicago   Illinois  19328  ...  January 27, 2017
```

8.3.1 Another HTML Example — Wikipedia Table with Filters

```
url = "https://en.wikipedia.org/wiki/Mobile_country_code"
df = pd.read_html(
    url,
    match="Country",        # Only extract tables that contain the word "Country"
    header=0,               # First row is the header
    storage_options={"User-Agent": "Mozilla/5.0"}
)
df[0]
```

Result:

```
   Mobile country code          Country ISO 3166  ...  Remarks
0                  289      A Abkhazia   GE-AB    ...  MCC is not listed by ITU
1                  412      Afghanistan      AF   ...  NaN
2                  276          Albania      AL   ...  NaN
...
251                648         Zimbabwe      ZW   ...  NaN
252 rows × 6 columns
```

8.4 Reading Data From Excel — `.read_excel()`

```
df_excel = pd.read_excel('Data_excel.xlsx')
df_excel
```

Result:

```
     Name  Age
0    Sher   19
1  Hassan   20
2    Khan   21
```

8.5 Pickle — Serializing Python Objects

Pickle is primarily used for **serializing and deserializing** a Python object structure. In other words, it converts a Python object into a byte stream to: - Store it in a file / database - Maintain program state across sessions - Transport data over the network

8.5.1 Converting a DataFrame to a Pickle File

```
df_excel.to_pickle('df_pickle')
# File saved with the name: df_pickle
```

> **Note:** There are many more methods available under `.read_xxx()` and `.to_xxx()` in pandas for various file formats.

---

9. Data Visualization With Matplotlib

> **"Matplotlib is a powerful plotting library for Python that enables the creation of static, animated, and interactive visualizations. It is widely used for data visualization in data science and machine learning."**

9.1 Importing Matplotlib

```
import matplotlib.pyplot as plt

# Note: Most of the data visualization functions are available in pyplot
```

9.2 Line Plot

Basic Line Plot

```
x = [1, 2, 3, 4, 5]  # X axis
y = [1, 4, 9, 16, 25]  # Y axis

# Creating a line plot
plt.plot(x, y)
plt.show()
```

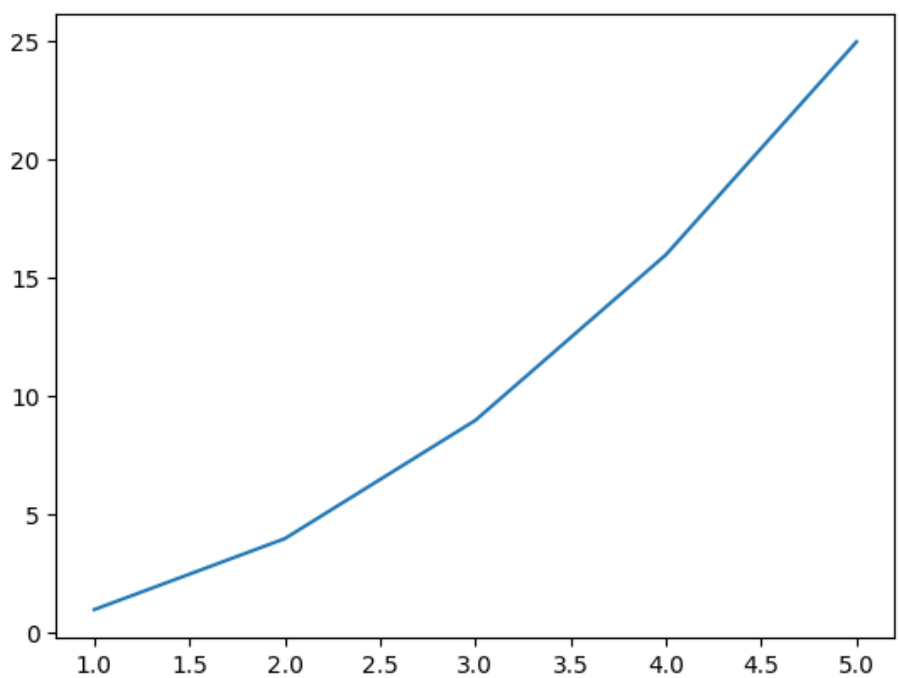

Figure 1: Basic Line Plot

Line Plot With Labels and Title

```
x = [1, 2, 3, 4, 5]  # X axis
y = [1, 4, 9, 16, 25]  # Y axis

plt.plot(x, y)
plt.xlabel('X Axis')        # X axis name
plt.ylabel('Y Axis')        # Y axis name
plt.title("Basic Line Plot") # Name of the graph
plt.show()                   # Show the graph
```

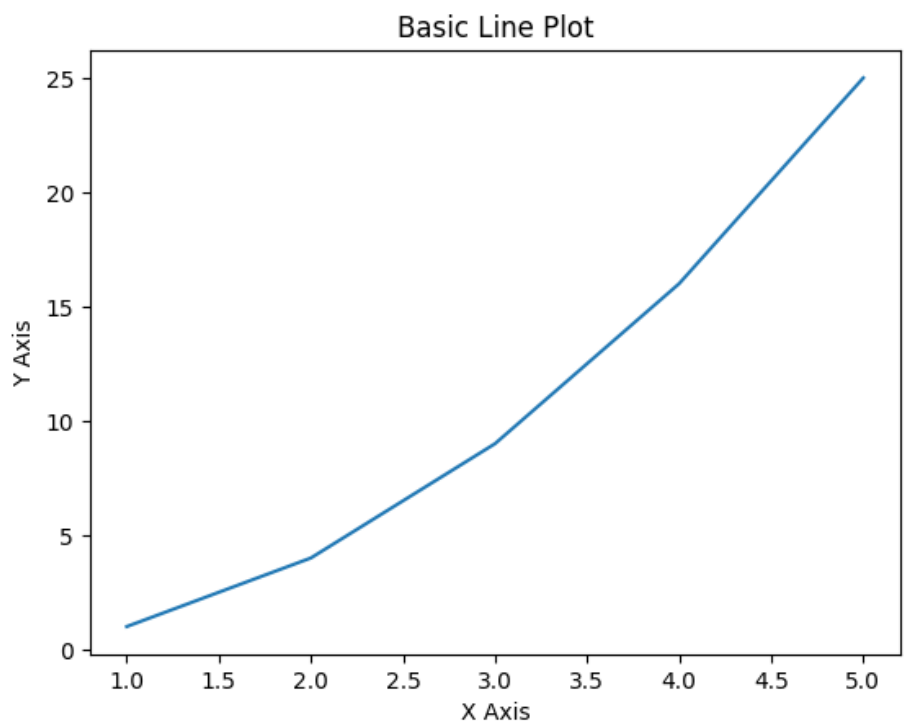

Figure 2: Line Plot with Labels

Customized Line Plot — Color and Linestyle

```
x = [1, 2, 3, 4, 5]
y = [1, 4, 9, 16, 25]

plt.plot(x, y, color='red', linestyle='--')
```

```
# Supported values for linestyle:
# '-', '--', '-.', ':', 'None', ' ', '',
# 'solid', 'dashed', 'dashdot', 'dotted'
```

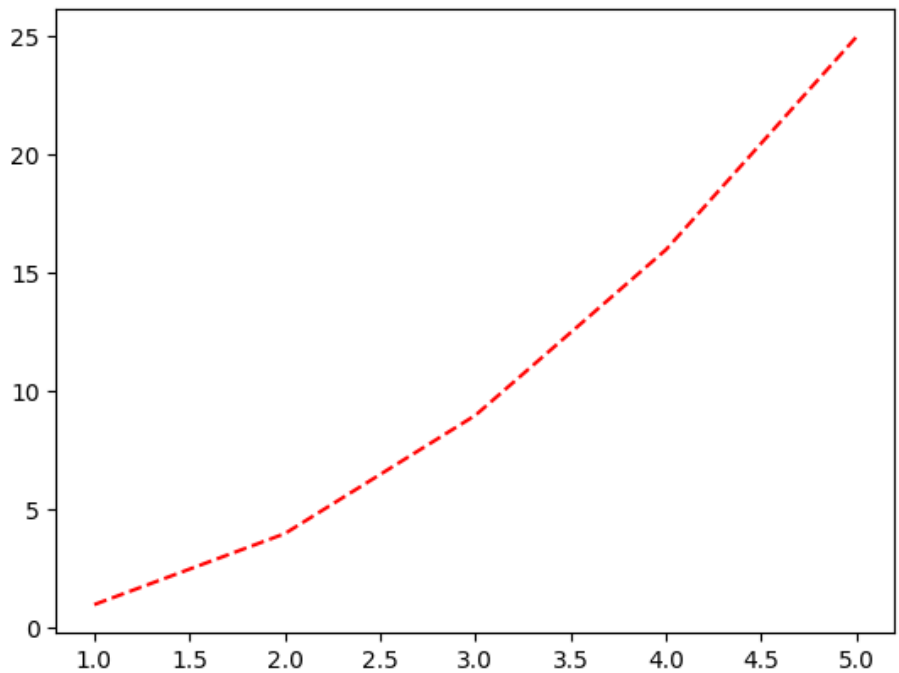

Figure 3: Customized Line Plot

Adding Markers

```
# Using the marker parameter
plt.plot(x, y, color='red', linestyle='--', marker='o')
# Puts 'o' on the values of x and y in the graph
# Other marker options: *, s, m, h, etc.
```

Line Width, Marker Size, and Grid

```
# line width and marker size parameters
plt.plot(x, y, color='red', linestyle='--', marker='h', linewidth=3, markersize=9)

# We can also add a grid
plt.grid(True)
```

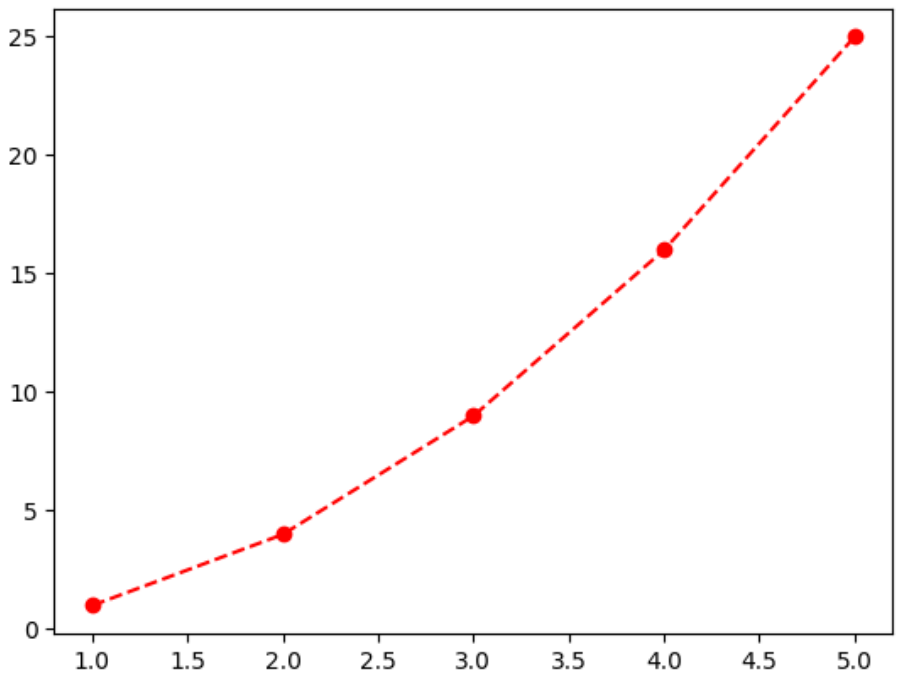

Figure 4: Line Plot with Markers

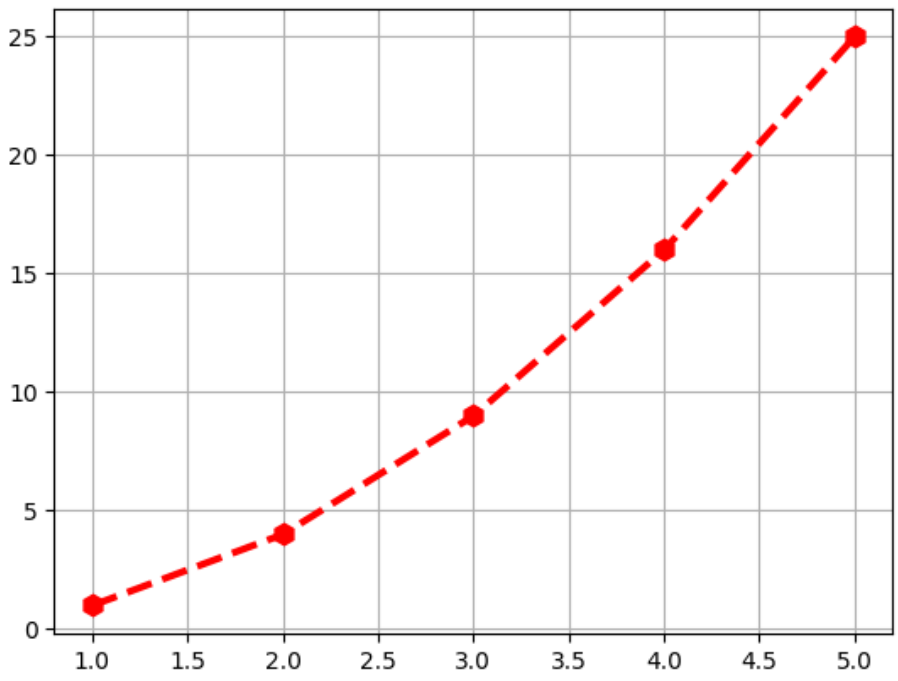

Figure 5: Line Plot with Width, Marker, Grid

9.3 Multiple Plots (Subplots)

```
# Sample data
x  = [1, 2, 3, 4, 5]
y1 = [1, 4, 9, 16, 25]
y2 = [1, 2, 3, 4, 5]

# Creating Parent Figure
plt.figure(figsize=(9, 5))
# (9,5) is the size of the figure in inches
# This is the main parent figure that includes independent subplots

# Plotting subplots
# plt.subplot(rows, cols, position)
# - First two args (2,2): how many rows and columns of subplots to create
# - Third arg: the position of this subplot in the parent figure

plt.subplot(2, 2, 1)
plt.plot(x, y1, color='green')
plt.title("plot 1")

plt.subplot(2, 2, 2)
plt.plot(y1, x, color='red')
plt.title("plot 2")

plt.subplot(2, 2, 3)
plt.plot(x, y2, color='blue')
plt.title("plot 3")

plt.subplot(2, 2, 4)
plt.plot(x, y2, color='yellow')
plt.title("plot 4")
```

9.4 Bar Plot

```
categories = ['A', 'B', 'C', 'D', 'E']
values = [5, 7, 3, 8, 6]

# Creating a Bar Plot
plt.bar(categories, values, color='purple')

# Additional informatory parameters
```

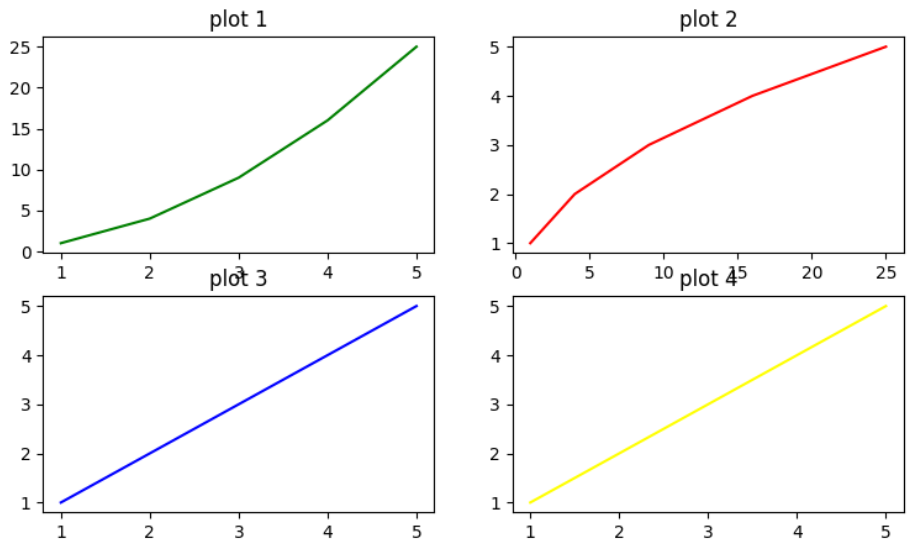

Figure 6: Multiple Subplots

```
plt.xlabel('X Axis')     # X axis name
plt.ylabel('Y Axis')     # Y axis name
plt.title("Bar Graph")   # Name of the graph
plt.show()
```

9.5 Histograms

> **"Histograms are used to represent the distribution of a dataset. They divide the data into bins and count the number of data points in each bin."**

```
# Sample data
data = [1, 2, 2, 3, 3, 3, 4, 4, 4, 4, 5, 5, 5, 5, 5]

# Creating a histogram
# bins=5 means: take this data and divide it into 5 bins
plt.hist(data, bins=5, color='orange', edgecolor="black")
```

Result:

```
(array([1., 2., 3., 4., 5.]),
 array([1. , 1.8, 2.6, 3.4, 4.2, 5. ]),
 <BarContainer object of 5 artists>)
```

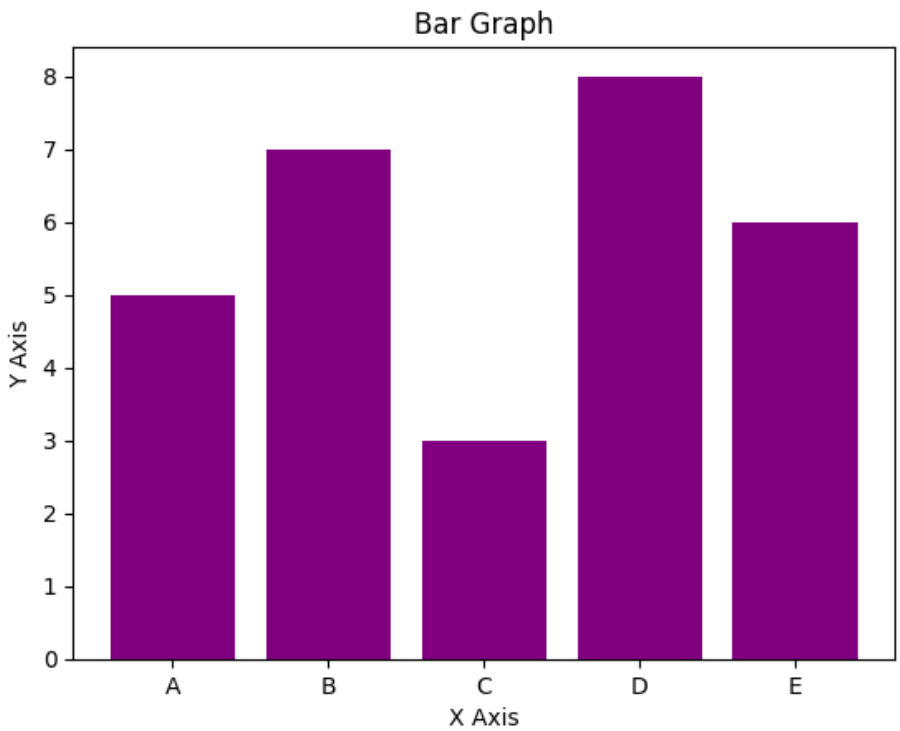

Figure 7: Bar Plot

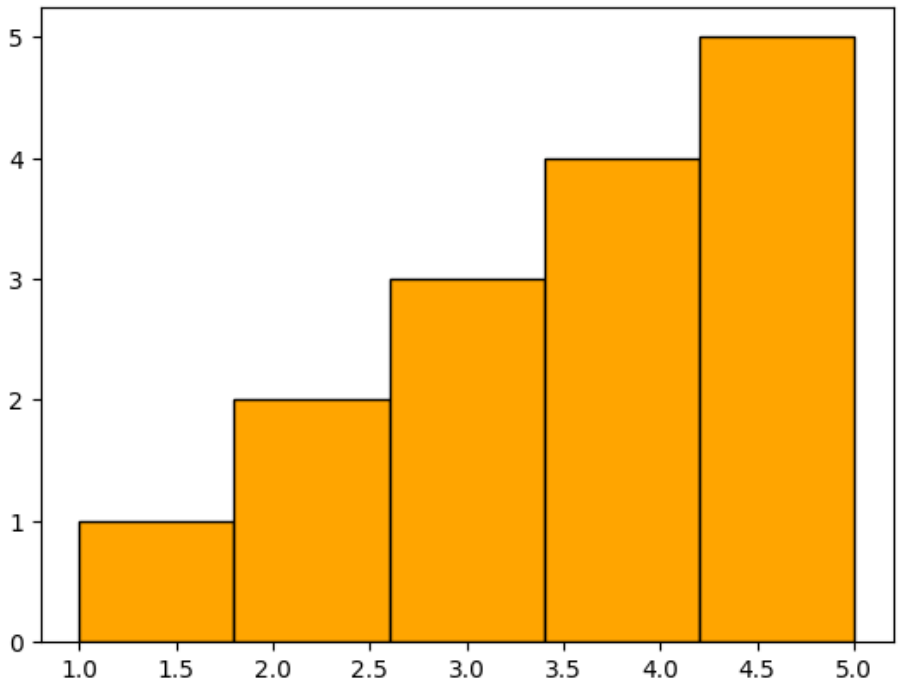

Figure 8: Histogram

9.6 Scatter Plot

> **"A scatter plot is a type of data visualization that uses dots to represent the values for two different variables. It is the go-to tool for identifying relationships or correlations between variables."**

```
# Sample Data
x = [1, 2, 3, 4, 5]
y = [2, 3, 4, 5, 6]

plt.scatter(x, y, color="blue", marker="x")
```

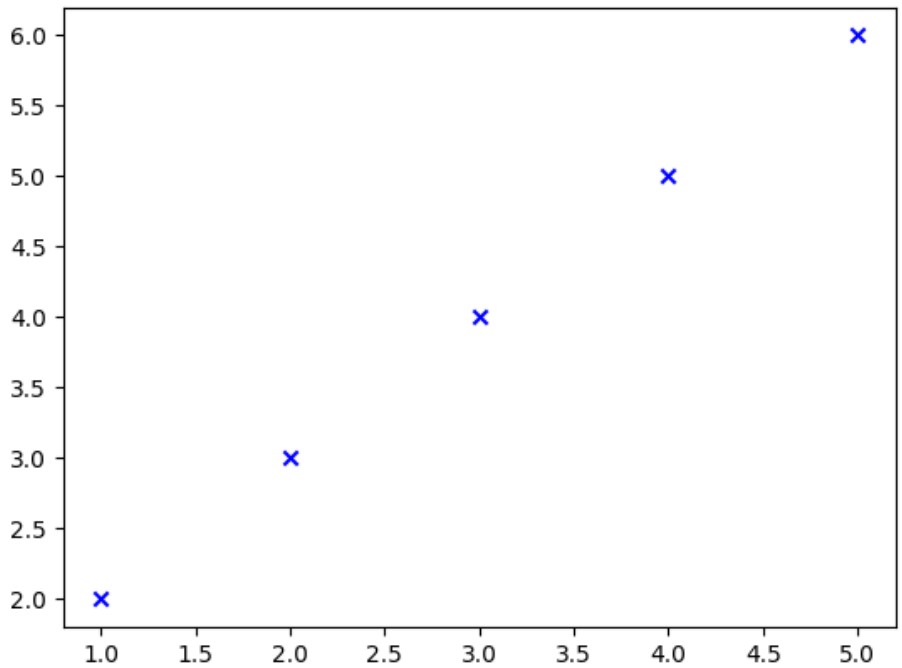

Figure 9: Scatter Plot

9.7 Pie Chart

```
labels  = ['A', 'B', 'C', 'D']
sizes   = [30, 20, 40, 10]
colors  = ['gold', 'yellowgreen', 'lightcoral', 'lightskyblue']
explode = (0.2, 0, 0, 0)  # Moves the first slice outward for decoration
```

```
plt.pie(
    sizes,
    explode=explode,
    labels=labels,
    colors=colors,
    autopct="%1.1f%%",    # Format: XX.X% (one decimal place)
    shadow=True
)
```

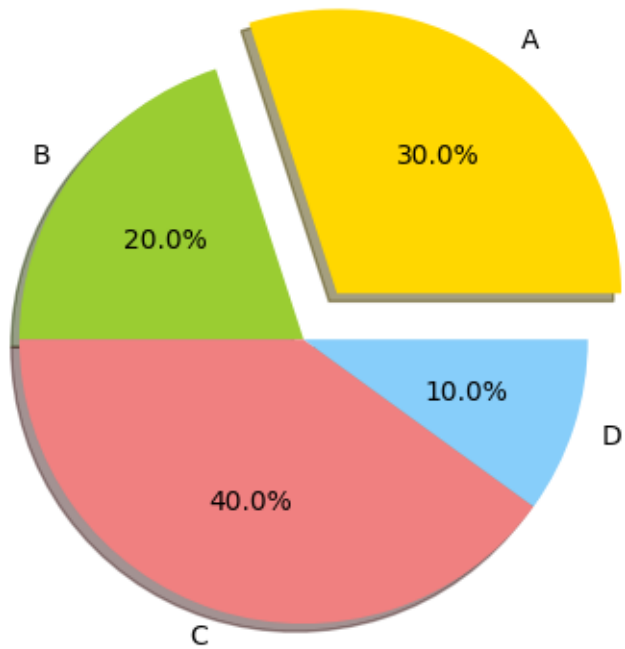

Figure 10: Pie Chart

9.8 Practical Example — Sales Data Visualization

```
import pandas as pd

sales_data_df = pd.read_csv('sales_data.csv')
sales_data_df.head()
```

9.8.1 Total Sales by Item Type (Bar Chart)

Let's group based on Item Type, sum the Total Revenue (which gives total sales), and then make a bar graph.

```
## Plot total sales by Item Type
sales_data_df.info()

total_sales_by_products = sales_data_df.groupby('Item Type')['Total Revenue'].sum()
print(total_sales_by_products)
```

Result (`sales_data_df.info()`):

```
<class 'pandas.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 14 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Region          100 non-null    str
 1   Country         100 non-null    str
 2   Item Type       100 non-null    str
 3   Sales Channel   100 non-null    str
 4   Order Priority  100 non-null    str
 5   Order Date      100 non-null    str
 6   Order ID        100 non-null    int64
 7   Ship Date       100 non-null    str
 8   Units Sold      100 non-null    int64
 9   Unit Price       96 non-null    float64
 10  Unit Cost       100 non-null    float64
 11  Total Revenue   100 non-null    float64
 12  Total Cost      100 non-null    float64
 13  Total Profit    100 non-null    float64
dtypes: float64(5), int64(2), str(7)
memory usage: 11.1 KB
```

```
# Creating the bar graph
total_sales_by_products.plot(kind='bar', color='teal')
```

9.8.2 Sales Trend Over Time (Line Chart)

```
# Plot sales trend over time

sales_trend = sales_data_df.groupby('Order Date')['Total Revenue'].sum().reset_index()
# reset_index() is used because after groupby, "Order Date" becomes the Index, not a column.
# We reset it so we can use it as the X axis directly.

sales_trend
```

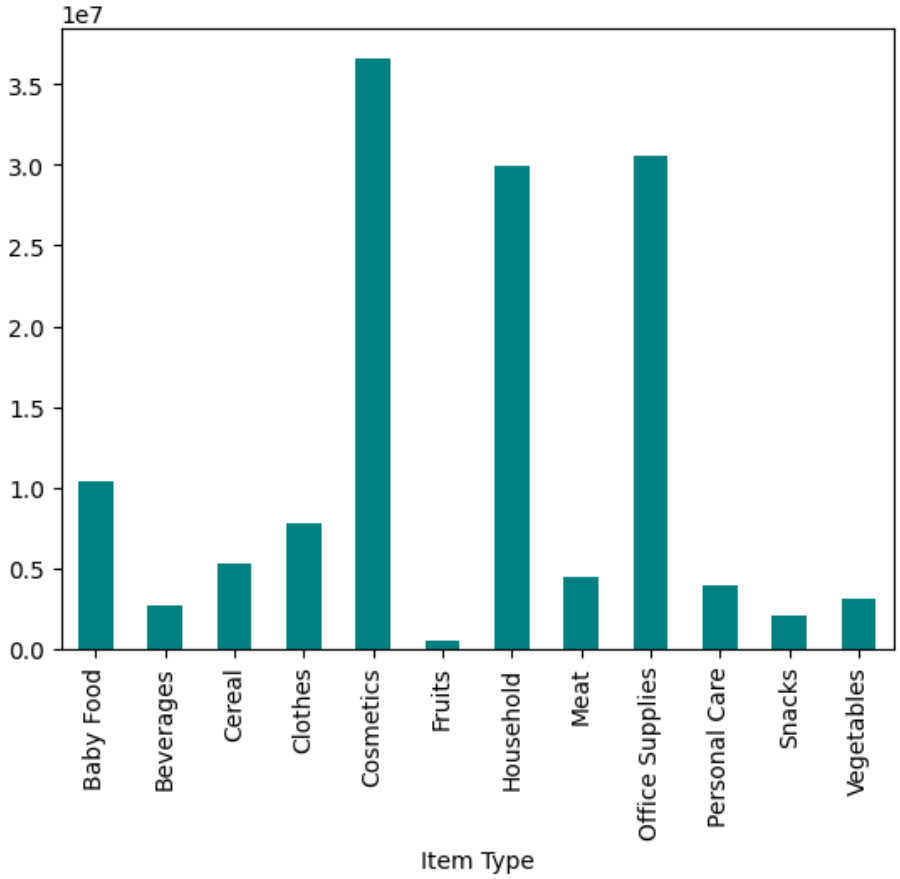

Figure 11: Total Sales by Item Type

Result (first few rows):

```
  Order Date    Total Revenue
0  1/11/2012      824431.86
1  1/13/2017      902980.64
2  1/14/2017     2011149.63
...
```

```
plt.plot(sales_trend['Order Date'], sales_trend['Total Revenue'])
```

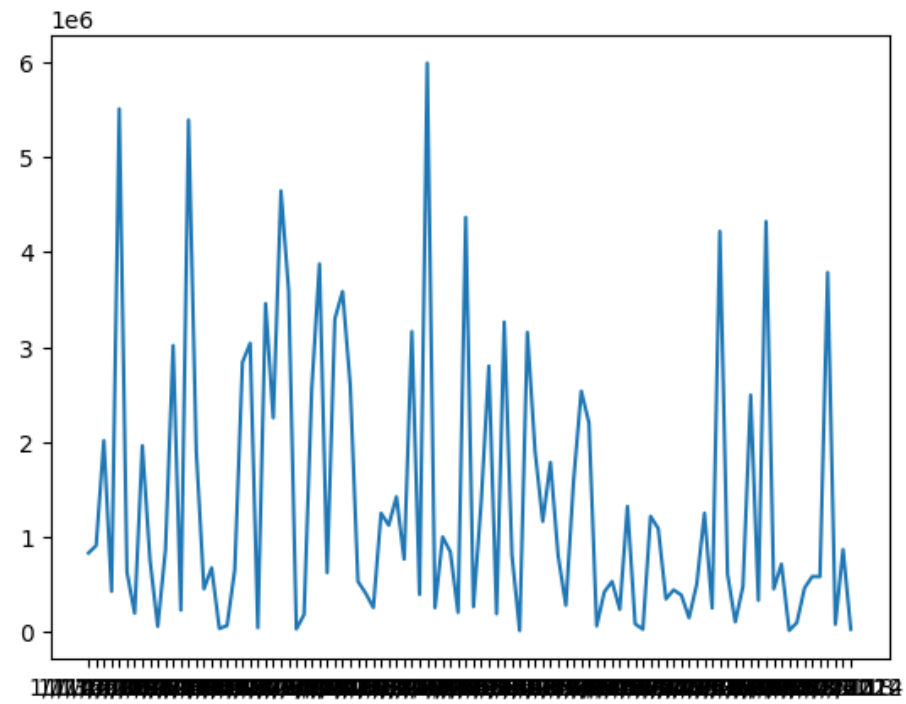

Figure 12: Sales Trend Over Time

---

10. Data Visualization With Seaborn

"Seaborn is a Python visualization library based on Matplotlib that provides a high-level interface for drawing attractive and informative statistical graphics. Seaborn helps in creating complex visualizations with just a few lines of code."

10.1 Importing Seaborn

```
import seaborn as sns
```

10.2 Built-in Datasets

Seaborn comes with a collection of **built-in sample datasets** that are used for practicing data visualization and testing library features.

```
sns.get_dataset_names()
```

Result:

```
['anagrams', 'anscombe', 'attention', 'brain_networks', 'car_crashes',
 'diamonds', 'dots', 'dowjones', 'exercise', 'flights', 'fmri',
 'geyser', 'glue', 'healthexp', 'iris', 'mpg', 'penguins', 'planets',
 'seaice', 'taxis', 'tips', 'titanic']
```

Loading a Dataset

```
tips = sns.load_dataset('tips')
print(type(tips))
tips
```

Result:

```
<class 'pandas.DataFrame'>
```

10.3 Common Syntax for Seaborn Plots

```
seaborn.plotname(x='Xname', y='Yname', data=DataFrameName)
```

10.4 Scatter Plot

```
import matplotlib.pyplot as plt

# sns.scatterplot(x='xname', y='yname', data=dataframename)
sns.scatterplot(x='total_bill', y='tip', data=tips)
plt.title("Scatter Plot of Total Bill vs Tip")
plt.show()
```

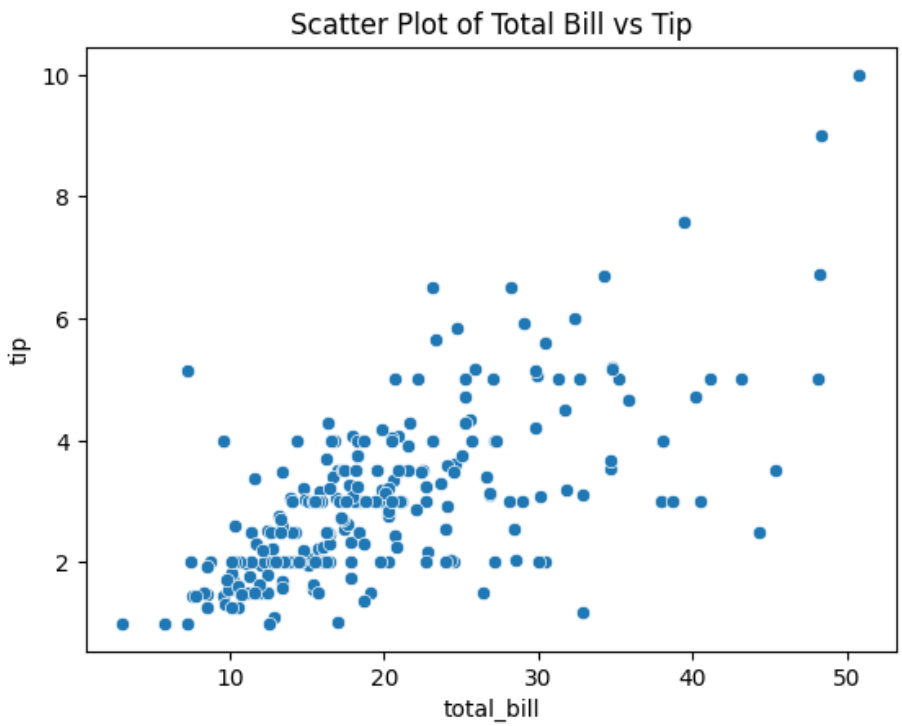

Figure 13: Scatter Plot of Total Bill vs Tip

10.5 Line Plot

```
# sns.lineplot(x='xname', y='yname', data=dataframename)
sns.lineplot(x='size', y='total_bill', data=tips)
plt.title("Line Plot of Total Bill by size")
plt.show()
```

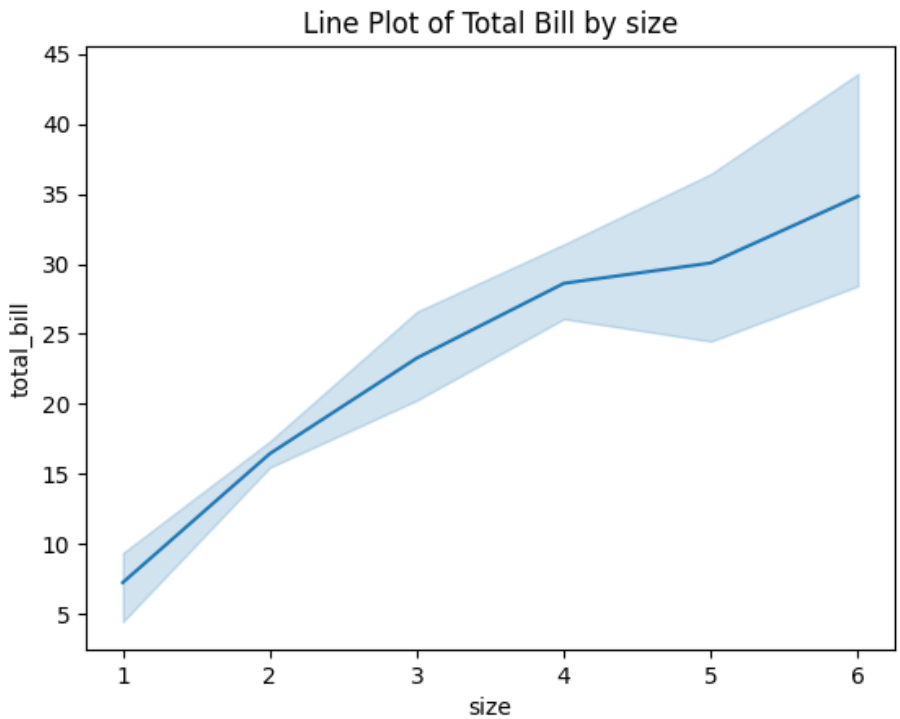

Figure 14: Line Plot of Total Bill by Size

10.6 Bar Plot (Categorical Plot)

```
sns.barplot(x='day', y='total_bill', data=tips)
plt.title("Bar plot of total bill by day")
plt.show()
```

10.7 Box Plot

```
sns.boxplot(x='day', y='total_bill', data=tips)
plt.show()
```

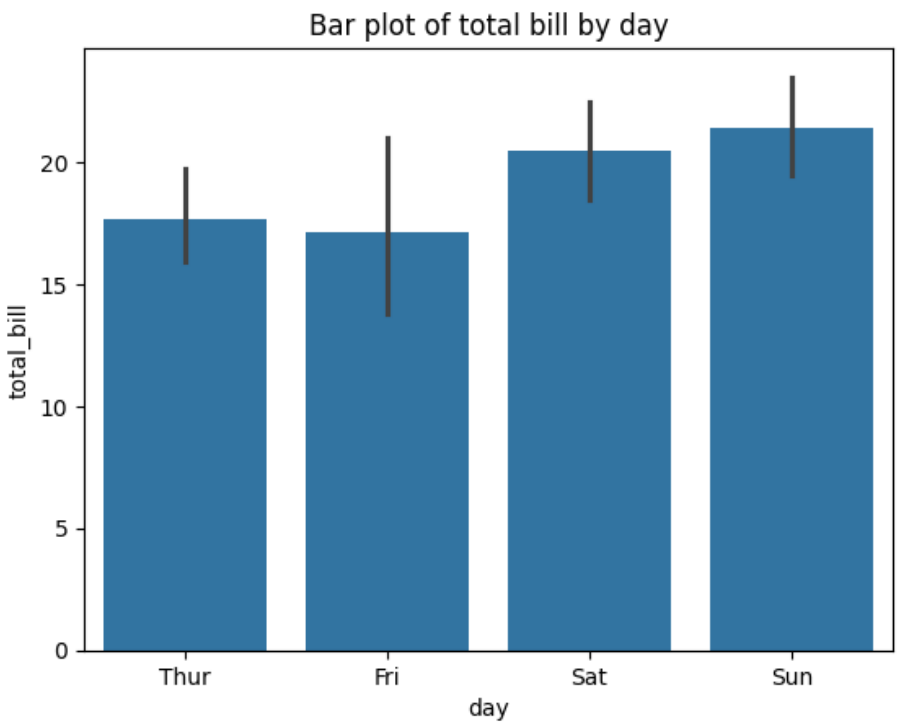

Figure 15: Bar Plot of Total Bill by Day

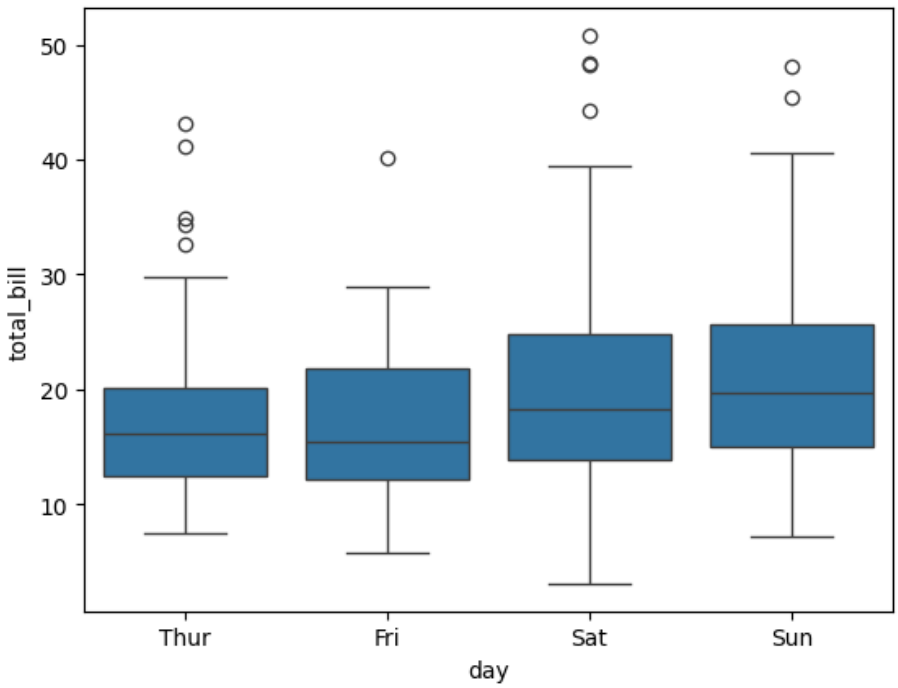

Figure 16: Box Plot of Total Bill by Day

10.8 Violin Plot

```
sns.violinplot(x='day', y='total_bill', data=tips)
plt.show()
```

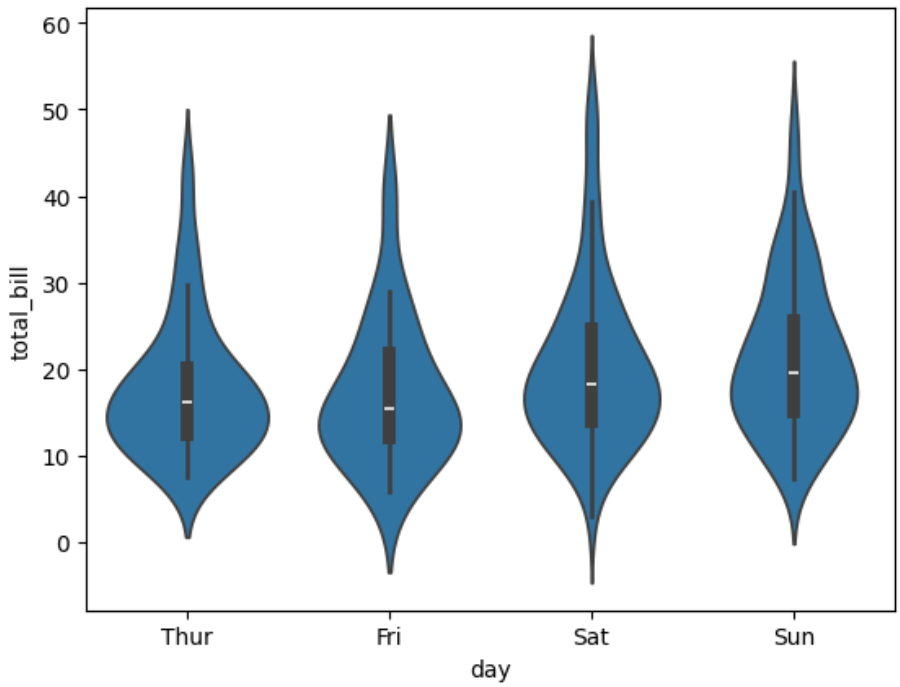

Figure 17: Violin Plot of Total Bill by Day

10.9 Histogram

```
sns.histplot(tips['total_bill'], bins=10, kde=True)
# kde=True adds the smooth curved line on top of the histogram
plt.show()
```

10.10 KDE Plot (Kernel Density Estimate)

```
sns.kdeplot(tips['total_bill'], fill=True)
plt.show()
```

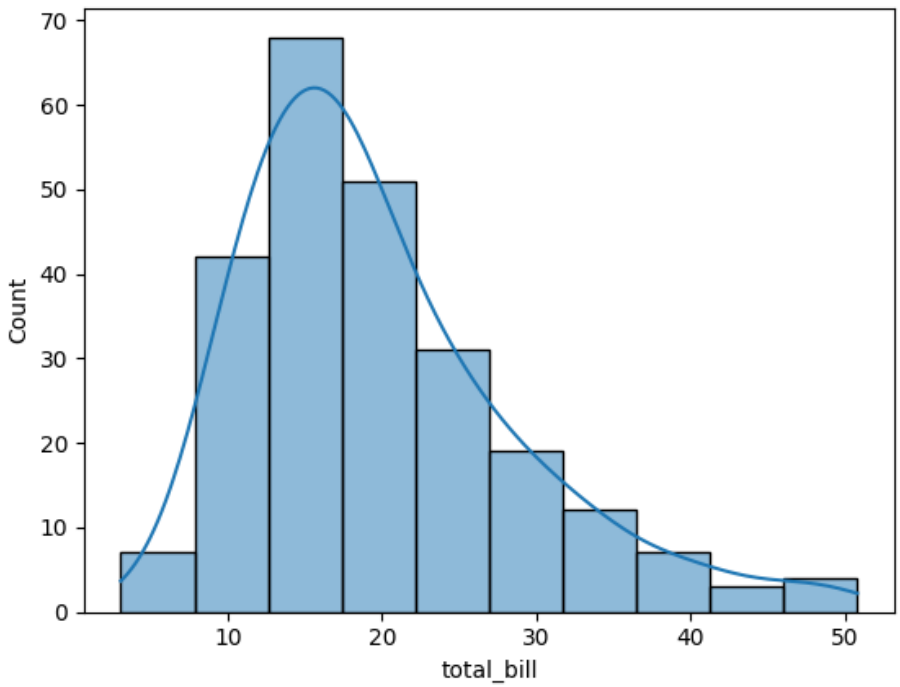

Figure 18: Histogram of Total Bill

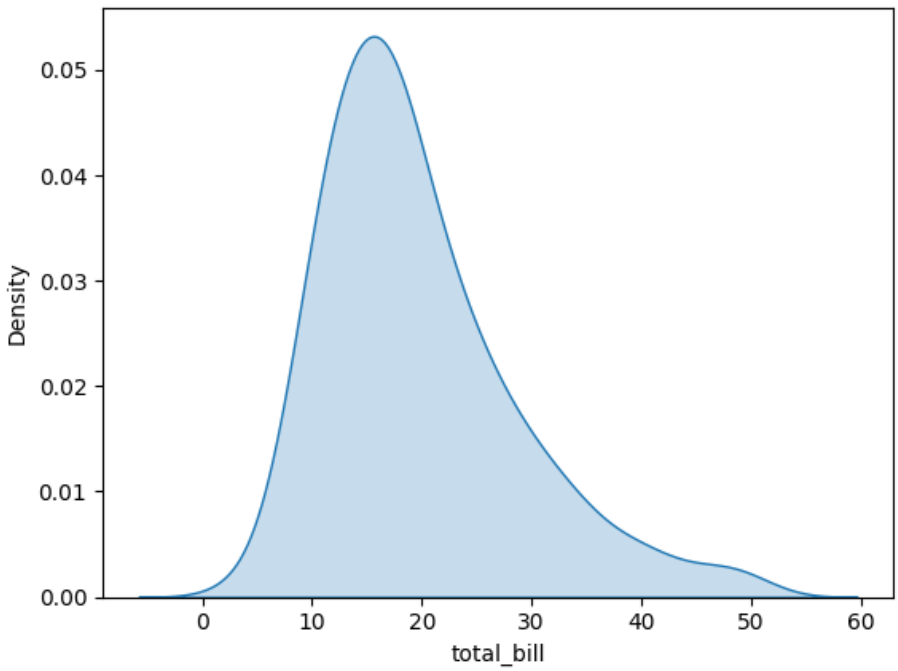

Figure 19: KDE Plot of Total Bill

10.11 Pair Plot

```
# Shows the relationship of all numerical columns with each other
sns.pairplot(tips)
plt.show()
```

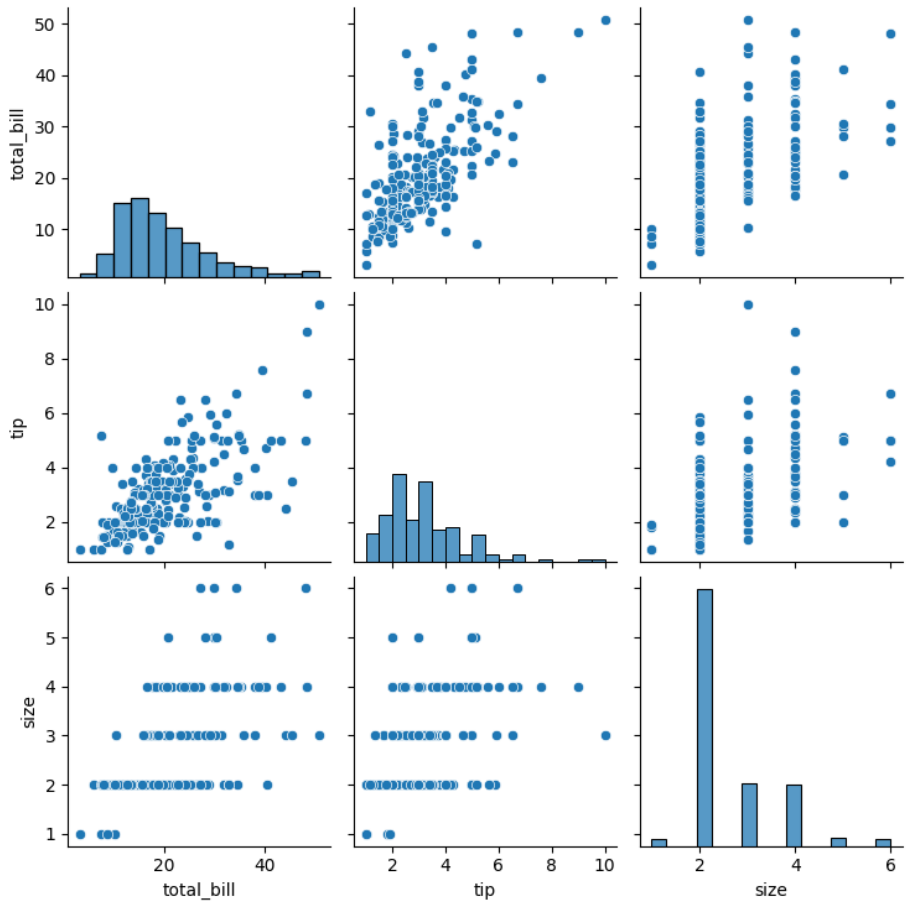

Figure 20: Pair Plot of Tips Dataset

10.12 Heatmap

A heatmap is used to visualize the **correlation** between numerical columns.

Step 1 — Compute the Correlation Matrix

```
corr = tips[['total_bill', 'tip', 'size']].corr()
# .corr() computes the pairwise correlation of numerical columns
# Only works on numerical data
corr
```

Step 2 — Plot the Heatmap

```
sns.heatmap(corr, annot=True, cmap='coolwarm')
# annot=True displays the correlation values inside each cell
# cmap='coolwarm' sets the color scheme
```

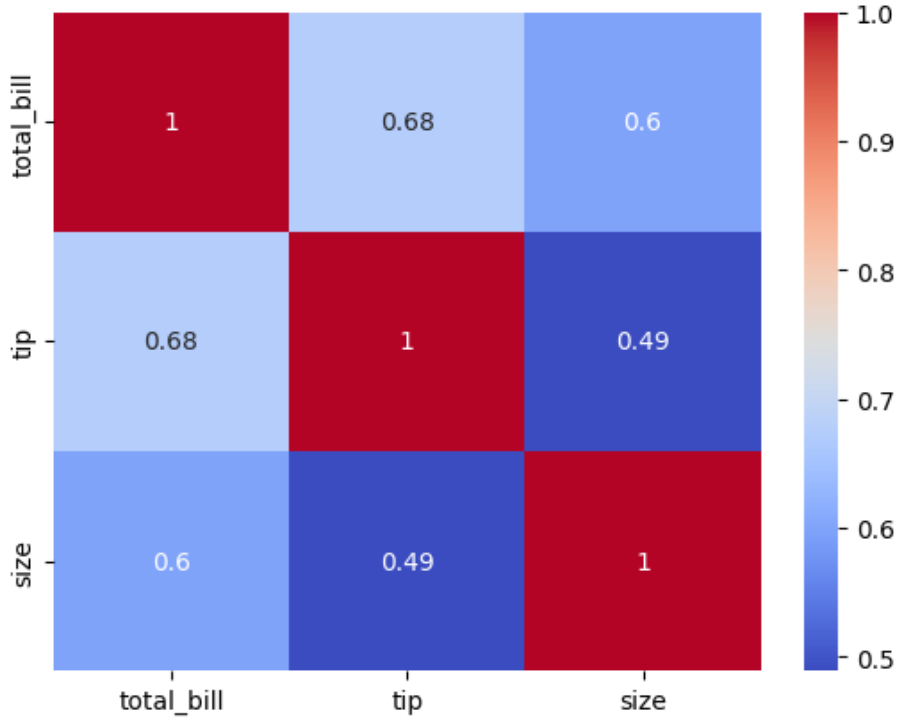

Figure 21: Heatmap of Correlation Matrix

10.13 Practical Example — Sales Data Visualization

```
import pandas as pd

myData = pd.read_csv('sales_data.csv')
```

```
myData.head()

# Using a bar graph to show Total Revenue by Item Type

plt.figure(figsize=(14, 6))                                  # First create the figure
sns.barplot(x='Item Type', y='Total Revenue', data=myData)  # Then plot the subplot
plt.title('Total revenue by Category')                       # Then add title
plt.xlabel('Category')                                       # Then label X axis
plt.ylabel('Revenue')                                        # Then label Y axis
plt.show()                                                   # Then show it
```

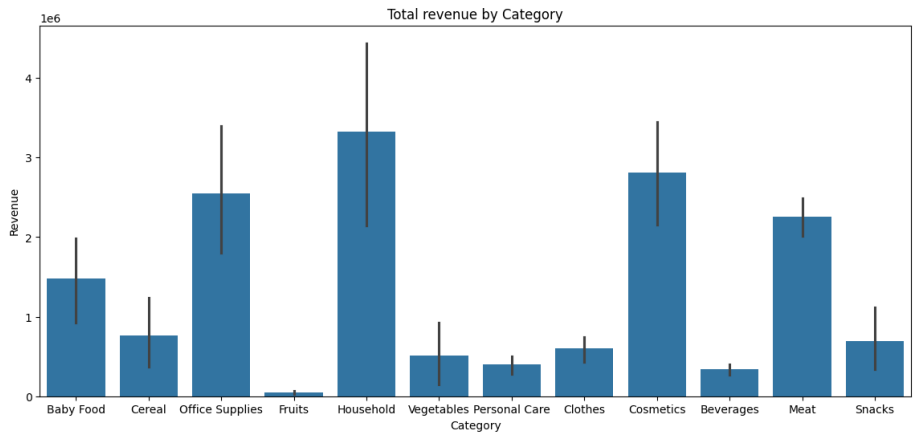

Figure 22: Total Revenue by Item Type

---

11. SQL and SQLite With Python

> **"SQL (Structured Query Language) is a standard language for managing and manipulating relational databases. SQLite is a self-contained, serverless, and zero-configuration database engine that is widely used for embedded database systems."**

11.1 Importing SQLite3

```
import sqlite3
```

11.2 Connecting to a Database

```
connection = sqlite3.connect('example.db')
connection
```

Result:

```
<sqlite3.Connection at 0x22023994c70>
```

Note: If `example.db` does not exist, SQLite will automatically create it.

11.3 Creating a Cursor

A cursor acts as a control structure or **"pointer"** that allows you to execute SQL queries and fetch the resulting rows one by one from the database.

```
cursor = connection.cursor()
```

11.4 Creating a Table

```
cursor.execute('''
    CREATE TABLE IF NOT EXISTS employees (
        id         INTEGER PRIMARY KEY,
        name       TEXT    NOT NULL,
        age        INTEGER,
        department TEXT
    )
''')

# Commit the changes
connection.commit()
```

Note: `IF NOT EXISTS` ensures the table is only created if it doesn't already exist — prevents errors on re-runs.

11.5 General Query Syntax

Any SQL query is executed by passing it as a string to `cursor.execute()`.

```
cursor.execute('''
    (SQL Query here)
''')
```

11.6 Inserting Data

Inserting Single Rows

```
cursor.execute('''
    INSERT INTO employees (name, age, department)
    VALUES ('Sher', 20, 'Data Scientist')
''')

cursor.execute('''
    INSERT INTO employees (name, age, department)
    VALUES ('John', 25, 'Data Engineer')
''')

cursor.execute('''
    INSERT INTO employees (name, age, department)
    VALUES ('Jane', 30, 'Machine Learning')
''')

# Commit the changes
connection.commit()
```

11.7 Querying Data — SELECT

```
cursor.execute('SELECT * FROM employees')
rows = cursor.fetchall()    # Get all the records (rows)

for row in rows:
    print(row)
```

Result:

```
(1, 'Sher', 20, 'Data Scientist')
(2, 'John', 25, 'Data Engineer')
(3, 'Jane', 30, 'Machine Learning')
```

11.8 Updating Data — UPDATE

```
cursor.execute('''
    UPDATE employees
    SET age = 27
    WHERE name = "Sher"
''')

connection.commit()
```

Verifying the Update

```
cursor.execute('SELECT * FROM employees')
rows = cursor.fetchall()

for row in rows:
    print(row)
```

Result:

```
(1, 'Sher', 27, 'Data Scientist')
(2, 'John', 25, 'Data Engineer')
(3, 'Jane', 30, 'Machine Learning')
```

11.9 Deleting Data — `DELETE`

```
cursor.execute('''
    DELETE FROM employees
    WHERE name = "John"
''')

connection.commit()
```

Verifying the Deletion

```
cursor.execute('SELECT * FROM employees')
rows = cursor.fetchall()

for row in rows:
    print(row)
```

Result:

```
(1, 'Sher', 27, 'Data Scientist')
(3, 'Jane', 30, 'Machine Learning')
```

11.10 Inserting Bulk Data — `executemany()`

Instead of calling `execute()` repeatedly, use `executemany()` to insert multiple rows at once. The `?` placeholders are filled in from each tuple in the list.

```
employees_data = [
    (1,  'Sher',    27, 'Data Scientist'),
    (2,  'jade',    29, 'Soft. Engineer'),
    (3,  'Jane',    30, 'Machine Learning'),
    (4,  'Alice',   24, 'Data Analyst'),
```

```
    (5,  'Bob',     29, 'Data Engineer'),
    (6,  'Charlie', 31, 'Machine Learning Engineer'),
    (7,  'Diana',   26, 'Business Analyst'),
    (8,  'Ethan',   35, 'DevOps Engineer'),
    (9,  'Fiona',   28, 'Software Engineer'),
    (10, 'George',  33, 'Product Manager'),
    (11, 'Hannah',  30, 'QA Engineer'),
    (12, 'Ivan',    27, 'Database Administrator')
]

cursor.executemany('''
    INSERT INTO employees (id, name, age, department)
    VALUES (?, ?, ?, ?)
''', employees_data)

connection.commit()
```

Verifying the Bulk Insert

```
cursor.execute('SELECT * FROM employees')
rows = cursor.fetchall()

for row in rows:
    print(row)
```

Result:

```
(1,  'Sher',    27, 'Data Scientist')
(2,  'jade',    29, 'Soft. Engineer')
(3,  'Jane',    30, 'Machine Learning')
(4,  'Alice',   24, 'Data Analyst')
(5,  'Bob',     29, 'Data Engineer')
(6,  'Charlie', 31, 'Machine Learning Engineer')
(7,  'Diana',   26, 'Business Analyst')
(8,  'Ethan',   35, 'DevOps Engineer')
(9,  'Fiona',   28, 'Software Engineer')
(10, 'George',  33, 'Product Manager')
(11, 'Hannah',  30, 'QA Engineer')
(12, 'Ivan',    27, 'Database Administrator')
```

11.11 Closing the Connection

Always close the connection when you are done to free up resources.

```
connection.close()
```

Quick Reference — SQLite3 Workflow

Step	Code	Purpose
Connect	`sqlite3.connect('file.db')`	Open / create a database
Cursor	`connection.cursor()`	Create a query pointer
Create Table	`cursor.execute('CREATE TABLE ...')`	Define table structure
Insert One	`cursor.execute('INSERT INTO ...')`	Add a single row
Insert Many	`cursor.executemany('INSERT INTO ...', data)`	Add multiple rows at once
Query	`cursor.execute('SELECT * FROM ...')`	Run a SELECT query
Fetch	`cursor.fetchall()`	Retrieve all result rows
Update	`cursor.execute('UPDATE ... SET ...')`	Modify existing rows
Delete	`cursor.execute('DELETE FROM ...')`	Remove rows
Commit	`connection.commit()`	Save all changes
Close	`connection.close()`	Free resources

---

12. Python Logging

> **"Logging is a crucial aspect of any application, providing a way to track events, errors, and operational information. Python's built-in logging module offers a flexible framework for emitting log messages from Python programs."**

12.1 Importing the Logging Module

```
import logging
```

12.2 Log Levels

Python's `logging` module has several log levels indicating the **severity** of events. Listed from lowest to highest severity:

Level	Numeric Value	Description
`DEBUG`	10	Detailed information, typically of interest only when diagnosing problems
`INFO`	20	Confirmation that things are working as expected
`WARNING`	30	Something unexpected happened, or indicative of a problem in the near future (e.g. "disk space low"). Software is still working as expected
`ERROR`	40	Due to a more serious problem, the software has not been able to perform some function
`CRITICAL`	50	A very serious error, indicating that the program itself may be unable to continue running

> **Note:** When you set a level (e.g. `DEBUG`), all messages at **that level and above** will be shown. Setting `WARNING` means `DEBUG` and `INFO` messages will be suppressed.

12.3 Basic Logging Setup

```python
# Configure the basic logging settings
logging.basicConfig(level=logging.DEBUG)  # There are other levels too

# Log messages with different severity levels
logging.debug("This is a debug message")
logging.info("This is an info message")
logging.warning("This is a warning message")
logging.error("This is an error message")
logging.critical("This is a critical message")
```

> **Result:**
>
> ```
> DEBUG:root:This is a debug message
> INFO:root:This is an info message
> ```

```
WARNING:root:This is a warning message
ERROR:root:This is an error message
CRITICAL:root:This is a critical message
```

Note: The format of each log line by default is `LEVEL:logger_name:message`. The logger name `root` is the default top-level logger.

12.4 Configuring Logging Format and Output File

You can customize how log messages look and where they are saved using `basicConfig()` with additional parameters.

```
logging.basicConfig(
    filename='app.log',                          # Save logs to this file
    filemode='w',                                # 'w' overwrites the file each run; use 'a
    level=logging.DEBUG,
    format='%(asctime)s - %(name)s - %(message)s',
    datefmt='%Y-%m-%d %H:%M:%S',
    force=True    # Required if basicConfig() was already called earlier in the session
                  # (logging does not allow re-configuration without force=True)
)

# Checking the output
logging.debug("This is a debug message")
logging.info("This is an info message")
logging.warning("This is a warning message")
logging.error("This is an error message")
logging.critical("This is a critical message")
```

`format` placeholders:

```
%(asctime)s    - Timestamp of when the log was created
%(name)s       - Name of the logger (default: root)
%(levelname)s  - Log level name (DEBUG, INFO, WARNING, etc.)
%(message)s    - The actual log message
```

`datefmt` uses standard `strftime` codes, e.g. `'%Y-%m-%d %H:%M:%S'` → `2024-01-15 09:30:00`

12.5 Configuring Logging with Handlers

Handlers are the components that determine **where** your log messages actually go. You can attach multiple handlers to send logs to different destinations simultaneously.

```
logging.basicConfig(
    level=logging.DEBUG,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    datefmt='%Y-%m-%d %H:%M:%S',
    handlers=[
        logging.FileHandler("app1.log"),   # Writes log messages to app1.log file
        logging.StreamHandler()            # Outputs log messages to the console (stdout)
    ]
)
```

> **Common Handlers:** - `FileHandler(filename)` — writes logs to a file - `StreamHandler()` — writes logs to the terminal / console - Both can be used together so logs appear in the terminal **and** get saved to a file at the same time

---

12.6 Practical Example — Organizing Logs Into a Folder

It is good practice to keep your logs organized in a dedicated `logs/` folder. Here is an example project structure:

```
Day 11
   logs
     __pycache__
       logger.cpython-314.pyc
     app.log
     logger.py       ← Central logger configuration
     test.py         ← File that uses the logger
   app.log
   LoggingPracticalImplementation.ipynb
```

`logger.py` — Central Logger Configuration

Create a reusable logger in a dedicated file so any module in your project can import it:

```
import logging
import os

# Ensure the logs directory exists
os.makedirs('logs', exist_ok=True)

logging.basicConfig(
    filename='logs/app.log',
    filemode='a',                    # Append mode - don't overwrite previous logs
    level=logging.DEBUG,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
```

```
    datefmt='%Y-%m-%d %H:%M:%S'
)

logger = logging.getLogger('MyAppLogger')
```

`test.py` — Using the Logger

```
from logger import logger

logger.debug("App started")
logger.info("Loading data...")
logger.warning("Data file is missing a column")
logger.error("Failed to connect to database")
logger.critical("Application is shutting down")
```

Quick Reference — `basicConfig()` Parameters

Parameter	Purpose	Example
`level`	Minimum severity level to log	`logging.DEBUG`
`filename`	Save logs to a file	`'app.log'`
`filemode`	File write mode	`'w'` (overwrite) or `'a'` (append)
`format`	Structure of each log line	`'%(asctime)s - %(levelname)s - %(message)s'`
`datefmt`	Format for the timestamp	`'%Y-%m-%d %H:%M:%S'`
`handlers`	List of output destinations	`[FileHandler(...), StreamHandler()]`
`force`	Re-apply config even if already set	`True`

---

13. Logging With Multiple Loggers / Modules

> **"You can create multiple loggers for different parts of your application."**

Instead of using the default `root` logger for everything, named loggers let each module have its **own identity, severity level, and behavior** — making logs much easier to trace and filter in larger projects.

13.1 Creating Multiple Loggers

Each logger is created with `logging.getLogger("name")` and can have its own level set independently.

```
import logging

# Create a logger for module1
logger1 = logging.getLogger("module1")
logger1.setLevel(logging.DEBUG)      # Captures DEBUG and above

# Create a logger for module2
logger2 = logging.getLogger("module2")
logger2.setLevel(logging.WARNING)    # Captures WARNING and above only

# Configure shared logging format
logging.basicConfig(
    level=logging.DEBUG,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    datefmt='%Y-%m-%d %H:%M:%S'
)
```

> **Note:** `basicConfig()` sets the global format and the root logger's level. Each named logger's own `.setLevel()` overrides this for that specific logger — so `logger2` will ignore `DEBUG` and `INFO` messages even though the root is set to `DEBUG`.

13.2 Logging Messages From Different Loggers

```
logger1.debug("This is a debug message for module 1")
logger2.warning("This is a warning message for module 2")
logger2.error("This is an error message for module 2")

# Now instead of 'root', the module name appears in each log line
```

> **Result:**
>
> ```
> 2026-03-18 09:54:33 - module1 - DEBUG   - This is a debug message for module 1
> 2026-03-18 09:54:33 - module2 - WARNING - This is a warning message for module 2
> 2026-03-18 09:54:33 - module2 - ERROR   - This is an error message for module 2
> ```
>
> **Notice:** `logger2.debug(...)` or `logger2.info(...)` would produce **no output** because `logger2`'s level is set to `WARNING` — anything below that is silently ignored.

13.3 How Named Loggers Work

Each call to `logging.getLogger("name")` returns the **same logger object** if the name has been used before — loggers are singletons identified by name. This means you can safely import and reuse the same logger across multiple files.

```
# In any file across your project:
logger = logging.getLogger("module1")   # Returns the same logger1 object as above
```

Logger Naming Convention

It is best practice to name loggers after the module they live in using `__name__`:

```
# Inside myapp/database.py
logger = logging.getLogger(__name__)
# __name__ resolves to 'myapp.database' automatically
```

This makes the logger name match the file path, so log output tells you exactly which file generated each message.

13.4 Practical Example — Multiple Loggers in a Project

```
import logging

# Shared format
logging.basicConfig(
    level=logging.DEBUG,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    datefmt='%Y-%m-%d %H:%M:%S'
)

# Logger for data loading module - verbose, captures everything
data_logger = logging.getLogger("data_loader")
data_logger.setLevel(logging.DEBUG)

# Logger for model training module - only important messages
model_logger = logging.getLogger("model_trainer")
model_logger.setLevel(logging.WARNING)

# Logger for API module - errors only
api_logger = logging.getLogger("api")
api_logger.setLevel(logging.ERROR)
```

```python
# Usage
data_logger.debug("Loading dataset from disk...")       # shown
data_logger.info("Dataset loaded: 1000 rows")           # shown
model_logger.debug("Starting epoch 1")                  # suppressed
model_logger.warning("Validation loss is increasing")   # shown
api_logger.info("Request received")                     # suppressed
api_logger.error("Failed to serialize response")        # shown
```

Result:

```
2026-03-18 09:54:33 - data_loader    - DEBUG   - Loading dataset from disk...
2026-03-18 09:54:33 - data_loader    - INFO    - Dataset loaded: 1000 rows
2026-03-18 09:54:33 - model_trainer  - WARNING - Validation loss is increasing
2026-03-18 09:54:33 - api            - ERROR   - Failed to serialize response
```

Quick Reference — Single vs Multiple Loggers

	Root Logger	Named Logger
How to get	`logging.warning(...)`	`logging.getLogger("name")`
Logger name in output	`root`	Your custom name
Level control	Global only	Per-logger, independent
Best for	Quick scripts	Multi-module applications
Reusable across files	No	Yes — same name returns same object

End of Notes